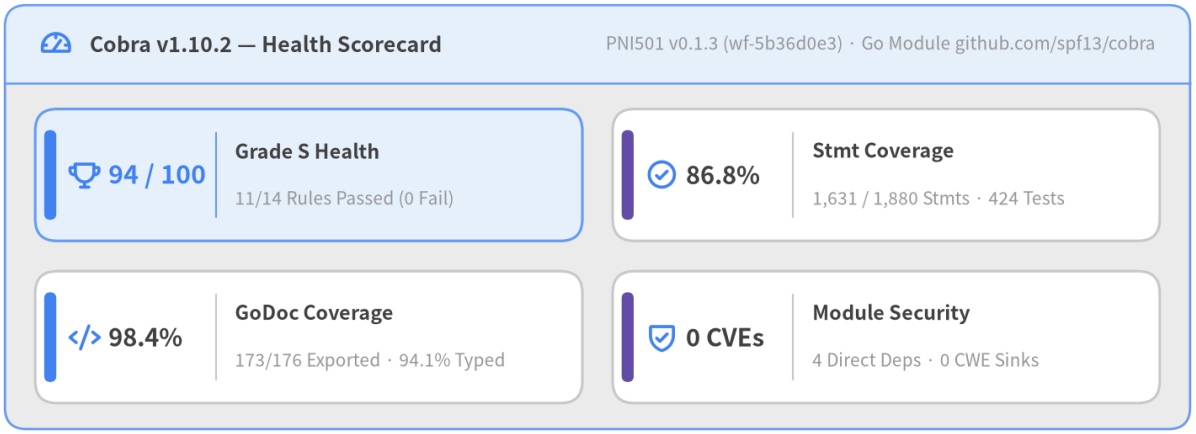
PNI Software Health Diagnostic Report

Cobra (`github.com/spf13/cobra`) `v1.10.2` — *Automated Go AST, Architecture, Hermetic Runtime & Supply-Chain Diagnostic*

This independent technical showcase evaluates **Cobra** `v1.10.2` (created by Steve Francia / `@spf13` and the Cobra Maintainers, Apache-2.0 License) across 7 software engineering dimensions using **PNI501** (`v0.1.3`) and **Drawlib**. All upstream Go source files, test suites, and module manifests (`go.mod` / `go.sum`) are inspected unmodified in zero-pollution hermetic environments.



Cobra v1.10.2 — PNI501 Overall Engineering Health Scorecard

Audit Attribute	Value
Target Project & Version	`github.com/spf13/cobra` `v1.10.2` (upstream tag `88b30ab8`, Apache-2.0 License)
Workflow & Run ID	`pni501` (`wf-5b36d0e3` , category: `audit` , PNI501 `v0.1.3`)
Language & Build System	Go (`go_mod` , `go.mod` + `go.sum` , tested in `golang:1.22-bookworm`)
Codebase Footprint	`19` production `.go` files (`4,888` LOC, `17` types, `269` functions) + `17` test files (`280` test funcs)
Hermetic Test & Coverage	`424` passed, `0` failed (`--network=none`), `86.8%` statement coverage (`1,631/1,880` stmts)
Overall Health Score	**94 / 100** (`Grade S`) — Security `98` , Testing `96` , Docs `95` , Arch `94` , Typing `93` , Deps `93` , Quality `90`

Audit Attribute	Value
Companion Documentation Portals	Cobra User Guide (`EN/JA`) · API Catalog · Internal Architecture

Table of Contents

1. Executive Summary

2. Architecture & Source Code Structure

3. Recommended Remediation Roadmap

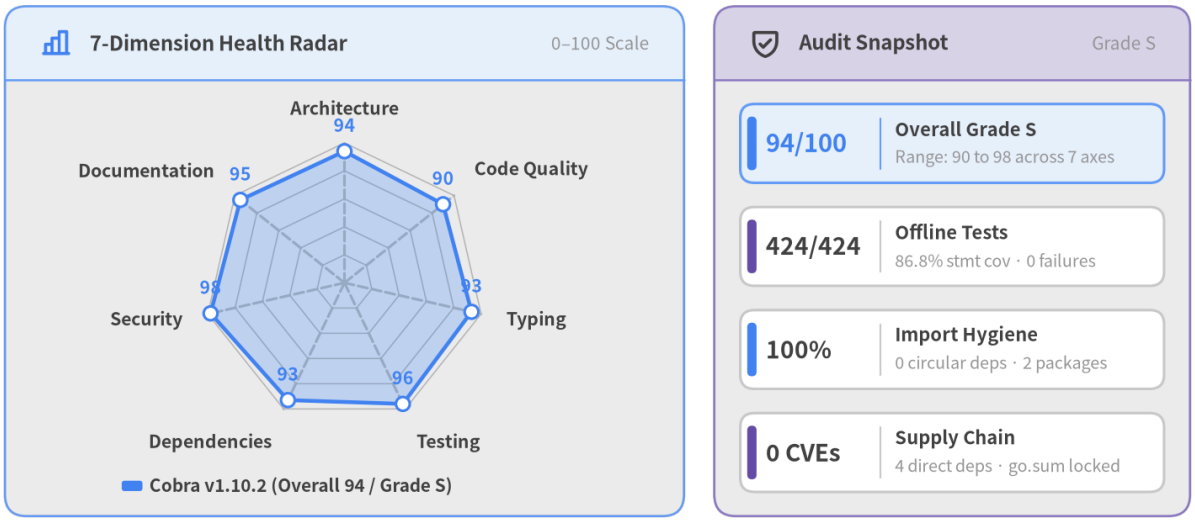
4. Audit Standards, Rule Compliance & Findings

5. Deep Dive: Code Quality, Testing & Runtime

6. Deep Dive: Security, Supply Chain & API Surface

1. Executive Summary

Cobra (`github.com/spf13/cobra`) `v1.10.2` achieves an overall PNI501 engineering health score of **94/100 (Grade S)** with **11/14** active rules passed (**3 WARN**, **0 FAIL**) across **19** Go production files (**4,888 LOC**, **17** exported types, **269** functions) and **17** co-located `*_test.go` suites (**280** test functions). Hermetic `--network=none` container execution confirms **424/424** passing test cases (**0** failures) and **86.8%** measured statement coverage (**1,631 / 1,880** statements), while Go AST and Trivy scans verify **98.4%** exported GoDoc coverage, **94.1%** type-safety coverage, **zero** circular dependencies, **zero** dangerous CWE execution sinks, **zero** hardcoded secrets, and **zero** CVEs across all Go module dependencies.



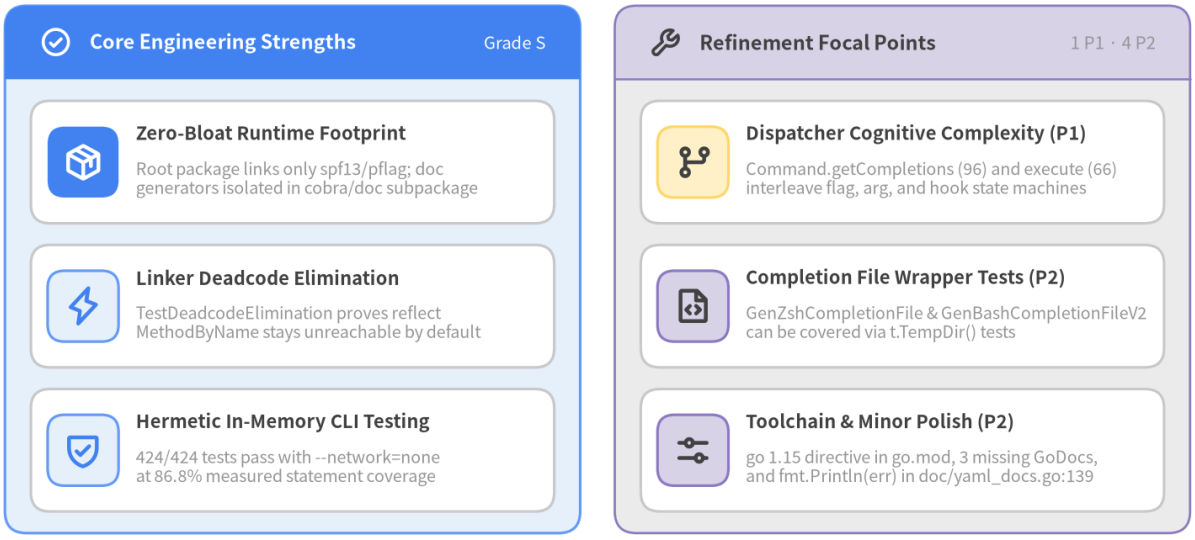
Cobra v1.10.2 — 7-Dimension PNI501 Engineering Health Radar & Headline Score Profile

1.1 Seven-Dimension Engineering Health Breakdown

Engineering Dimension	Score	Grade	Deterministic Evidence & Key Rationale
Security	**98 / 100**	`S`	`0` CVEs across all Go module dependencies (`app_vuln_counts.total = 0`), `0` CWE sinks (`dangerous_sinks = []`), `0` secrets, and `0` Dockerfile IaC misconfigurations.

Engineering Dimension	Score	Grade	Deterministic Evidence & Key Rationale
Testing	**96 / 100**	`S`	`17` co-located `*_test.go` files (`100.0%` module alignment, `280` test funcs), `424/424` offline tests passing (`7.03s`), and `86.8%` measured statement coverage (`1,631/1,880` stmts).
Documentation	**95 / 100**	`S`	`98.4%` exported GoDoc coverage (`173/176` documented, `0` empty, `0` signature mismatches) plus built-in `cobra/doc` Markdown, Man, ReST, and YAML generators.
Architecture	**94 / 100**	`S`	Acyclic 2-package layout (`github.com/spf13/cobra` + `github.com/spf13/cobra/doc`), `100.0%` import hygiene, `0` circular dependencies, and isolated doc rendering deps.
Typing	**93 / 100**	`S`	`94.1%` static type safety coverage across `17` exported types and `269` functions, `0` `unsafe.Pointer` usages, and `0` unscoped `//nolint` directives.
Dependencies	**93 / 100**	`S`	Ultra-lean 4-package direct dependency set (`pflag v1.0.9` , `go-md2man/v2 v2.0.6` , `mousetrap v1.1.0` , `yaml/v3 v3.0.4`) locked in `go.sum` ; `go.mod` retains `go 1.15` .
Code Quality	**90 / 100**	`S`	Clean idiomatic Go (`0` `go vet` warnings, `0` `>5` -param funcs); cognitive complexity concentrates in `Command.getCompletions` (`96`) and `Command.execute` (`66`).

1.2 Principal Engineering Strengths vs. Refinement Focal Points



Cobra v1.10.2 — Architectural Strengths vs. Targeted Engineering Refinement Focal Points

2. Architecture & Source Code Structure

Cobra `v1.10.2` organizes **19 Go production source files** (**4,888 LOC**, **17 exported types**, **269 functions**) and **17 test files** (**280 test functions**) into a clean **4-Layer Composite Command Tree & Extension Subsystem** architecture (**94/100**, Grade **S**). The root package (`github.com/spf13/cobra`) encapsulates the composite `Command` tree, `pflag` inheritance, relational flag groups, and the Shell Completion V2 engine while isolating Markdown, Man, ReST, and YAML reference generators inside the `github.com/spf13/cobra/doc` subpackage.

Cobra v1.10.2 — Physical Package Layout & Subsystem Inventory (19 Source Files · 4,888 LOC)

Go Module Tree — 0 Cycles

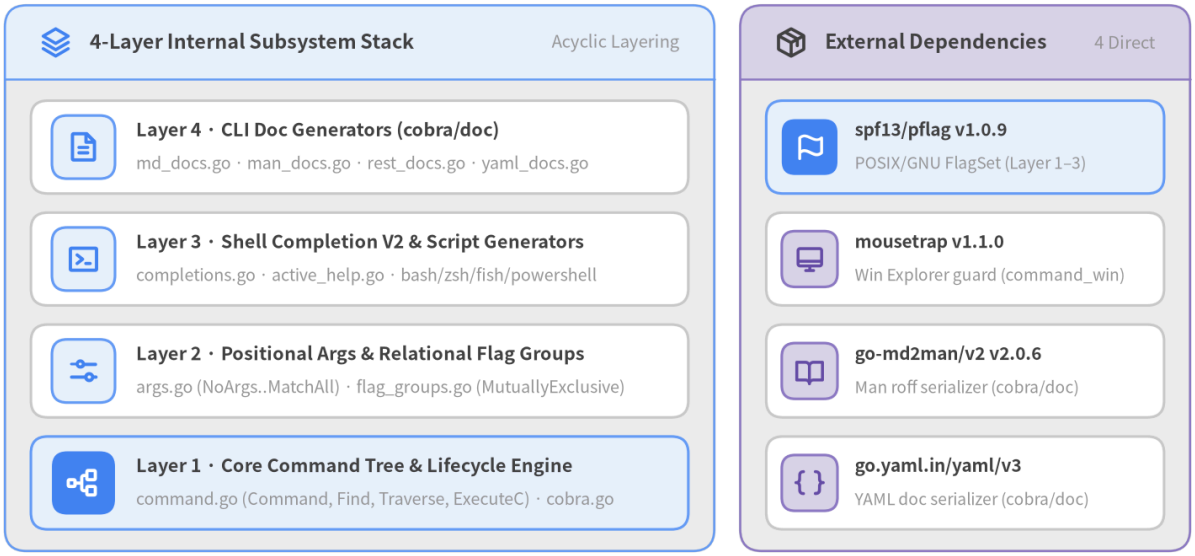
```
github.com/spf13/cobra (4,888 LOC)
├── command.go (1,417 LOC)
├── cobra.go (159 LOC)
├── args.go (83 LOC)
├── flag_groups.go (211 LOC)
├── completions.go (600 LOC)
├── active_help.go & shell (61 LOC)
├── bash_completions*.go (1,004)
├── zsh/fish/pwsh_*.go (723)
└── doc/ (5 files · 608 LOC)
    └── md/man/rest/yaml_docs.go
```

Subsystem Breakdown — 19 Files · 86.8% Cov

Subsystem	Files	LOC	Cov.	Primary Responsibility
Core Tree & Exec	4	1,598	84.1%	Command struct, Find, ExecuteC
Args & Flag Groups	2	294	96.4%	PositionalArgs & flag groups
Completion Engine	3	661	92.6%	__complete protocol & ActiveHelp
Shell Generators	5	1,727	86.1%	Bash V1/V2, Zsh, Fish & Pwsh
cobra/doc Package	5	608	86.5%	MD, Man, ReST & YAML doc trees
Total / Overall	19	4,888	86.8%	17 exported types · 269 funcs

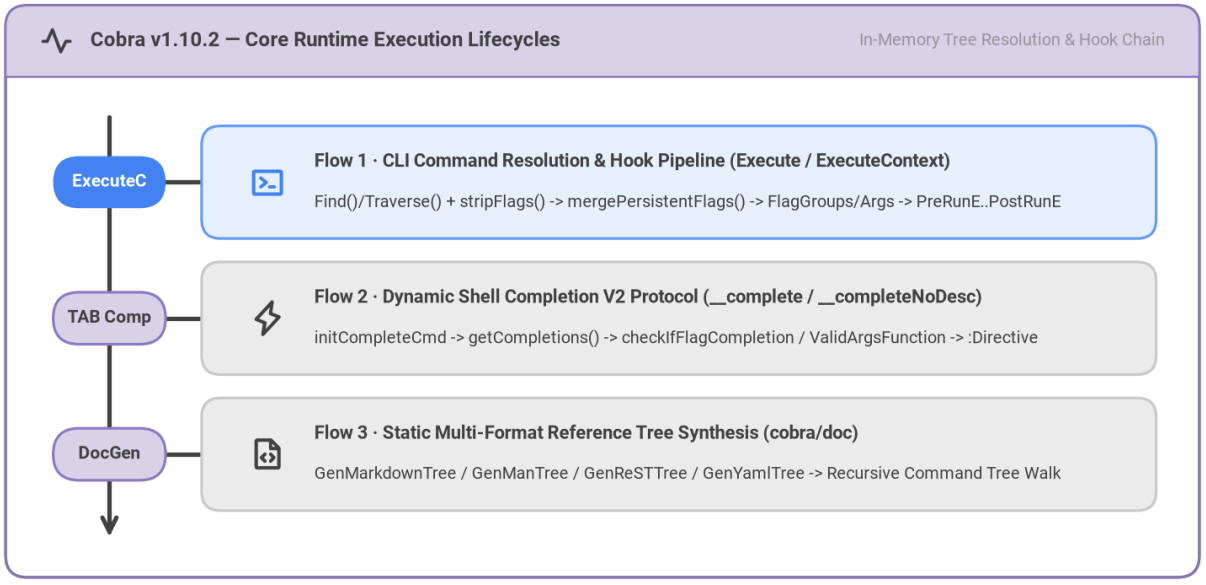
Cobra v1.10.2 Source Package Tree & Subsystem LOC Distribution (19 Production Files, 4,888 LOC)

2.1 Four-Layer Subsystem Dependency Topology



Cobra v1.10.2 Four-Layer Internal Architecture & External Dependency Isolation

2.2 Core Execution Flows & Runtime Lifecycles



Cobra v1.10.2 Core Runtime Lifecycles: ExecuteC Command Dispatch, __complete V2 Protocol & cobra/doc Synthesis

Three Canonical Execution Lifecycles

1. **Flow 1 — CLI Command Resolution & Hook Execution Pipeline (`Command.ExecuteC`)**:
   - `ExecuteContext(ctx)` binds `context.Context` to the root `Command` and invokes `ExecuteC()`.
   - `ExecuteC()` initializes default completion, help, and version commands/flags, then resolves `os.Args[1:]` via `Command.Find()` (or `Command.Traverse()` when `TraverseChildren: true`), using `stripFlags()` to skip flag tokens and flag values while descending the subcommand tree.
   - `Command.execute()` merges ancestor `PersistentFlags()` via `mergePersistentFlags()`, parses flags with `pflag`, runs `cobra.OnInitialize(...)` callbacks, validates required flags (`ValidateRequiredFlags`), relational flag groups (`ValidateFlagGroups`), and positional arguments (`ValidateArgs`), and executes hooks in strict order:

`PersistentPreRunE` -> `PreRunE` -> `RunE` -> `PostRunE` -> `PersistentPostRunE` -> `OnFinalize`.

2. **Flow 2 — Dynamic Shell Completion V2 Protocol (`__complete` / `__completeNoDesc`)**:
    - Shell scripts generated by `GenBashCompletionV2`, `GenZshCompletion`, `GenFishCompletion`, and `GenPowerShellCompletionWithDesc` invoke `<binary> __complete <args...>` when the user presses `TAB`.
    - `Command.getCompletions()` parses partial flags without invoking `RunE`, checks whether the cursor is completing a flag via `checkIfFlagCompletion()`, or enforces flag-group exclusions (`enforceFlagGroupsForCompletion`) and calls `ValidArgsFunction` / `ValidArgs`, emitting candidates plus a trailing `:<ShellCompDirective>` bitmask (`ShellCompDirectiveNoFileComp = 4`, etc.) and optional `_activeHelp_` hints.
3. **Flow 3 — AST-Free Multi-Format Documentation Synthesis (`cobra/doc`)**:
    - `GenMarkdownTree`, `GenManTree`, `GenReSTTree`, and `GenYamlTree` walk the live `Command` hierarchy, rendering `Use`, `Short`, `Long`, `Example`, local flags, inherited persistent flags, and sorted parent/child cross-links without requiring source-code AST parsing.

3. Recommended Remediation Roadmap

Cobra `v1.10.2` enters remediation planning from an exceptional **94/100 (Grade S)** baseline with **zero** `P0` **urgent blockers**, **zero** `FAIL` **compliance rules**, and **zero** `CRITICAL` **or** `HIGH` **source-code or CVE findings** across all 19 Go source files and 424 passing hermetic tests. The 5-item engineering roadmap focuses on **1** `P1` **high-leverage dispatcher complexity decomposition** followed by **4** `P2` **test-coverage, GoDoc, library hygiene, and** `go.mod` **toolchain polish items**.

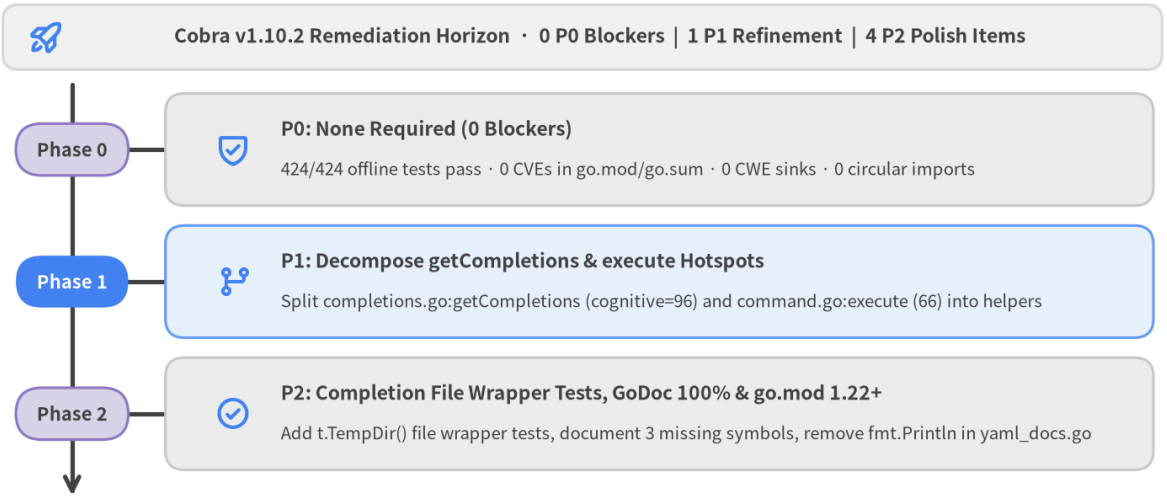
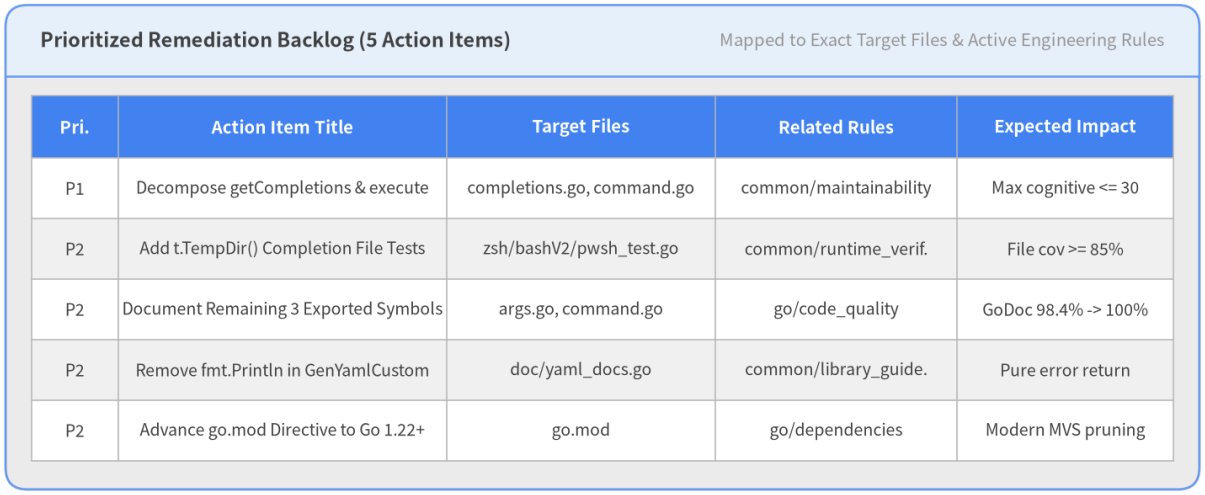


Figure 3.1 — Phased Engineering Refinement Roadmap for Cobra v1.10.2 (0 P0 Blockers · 1 P1 High-Leverage · 4 P2 Polish)

3.1 Actionable Engineering Backlog & Traceability Matrix



Prioritized Remediation Backlog (5 Action Items) — Mapped to Exact Target Files & Active Engineering Rules

Pri.	Action Item Title	Target Files	Related Rules	Expected Impact
P1	Decompose getCompletions & execute	completions.go, command.go	common/maintainability	Max cognitive <= 30
P2	Add t.TempDir() Completion File Tests	zsh/bashV2/pwsh_test.go	common/runtime_verif.	File cov >= 85%
P2	Document Remaining 3 Exported Symbols	args.go, command.go	go/code_quality	GoDoc 98.4% -> 100%
P2	Remove fmt.Println in GenYamlCustom	doc/yaml_docs.go	common/library_guide.	Pure error return
P2	Advance go.mod Directive to Go 1.22+	go.mod	go/dependencies	Modern MVS pruning

Figure 3.2 — Prioritized Action Plan Traceability Matrix Across Target Files & PNI501 Rules

Priority	Category	Action Title & Target Files	Concrete Engineering Action	Expected Impact
`P1`	`code_quality`	**Decompose `Command.getCompletions` and `Command.execute` Hotspots** (`completions.go` , `command.go` , `flag_groups.go`)	Factor flag-completion resolution, positional `ValidArgsFunction` invocation, and ActiveHelp synthesis out of `Command.getCompletions` (`cognitive=96`), and extract hook-chain traversal (`PersistentPreRunE` .. `PersistentPostRunE`) out of `Command.execute` (`cognitive=66`).	Reduces maximum cognitive complexity below `30` and simplifies maintenance of the CLI dispatch and completion engines.
`P2`	`testing`	**Add `t.TempDir()` Unit Tests for Completion File Wrapper Methods** (`zsh_completions_test.go` , `bash_completionsV2_test.go` , `powershell_completions_test.go`)	Add table-driven unit tests using `t.TempDir()` for `GenZshCompletionFile` , `GenZshCompletionFileNoDesc` , `GenBashCompletionFileV2` , and `GenPowerShellCompletionFile` .	Lifts per-file statement coverage in `zsh_completions.go` (`52.6%`) and `bash_completionsV2.go` (`57.1%`) above `85%` .
`P2`	`documentation`	**Add GoDoc Comments to Remaining 3 Exported Symbols** (`args.go` , `command.go` , `completions.go`)	Add standard leading GoDoc comments to `PositionalArgs` (`args.go:22`), `Command.ValidateArgs` (`command.go:1172`), and `CompletionOptions.SetDefaultShellCompDirective` (`completions.go:123`).	Raises exported GoDoc coverage from `98.4%` (`173/176`) to `100.0%` (`176/176`).

Priority	Category	Action Title & Target Files	Concrete Engineering Action	Expected Impact
`P2`	`code_quality`	**Remove Direct `fmt.Println(err)` Side Effect in `doc/yaml_docs.go`** (`doc/yaml_docs.go`)	Delete `fmt.Println(err)` at `doc/yaml_docs.go:139` before `return err` in `GenYamlCustom`.	Ensures pure library error propagation without unexpected stdout output.
`P2`	`dependencies`	**Modernize `go.mod` Toolchain Directive to Go 1.22+** (`go.mod`)	Advance `go 1.15` in `go.mod` to `go 1.22` (or `go 1.23`) once legacy consumer support windows close.	Enables modern Go module graph pruning and standard library generics/slices idioms.

4. Audit Standards, Rule Compliance & Findings

PNI501 (`v0.1.3`) evaluated Cobra `v1.10.2` across **6 deterministic audit steps** and **14 active engineering compliance rules** spanning Go module packaging, static Go AST maintainability, library API purity, table-driven `testing` hygiene, hermetic `--network=none` container execution, and Trivy supply-chain security. Every audit step achieved Grade `S` (`90/100` to `98/100`), yielding **11** `PASS` **rules**, **3 advisory** `WARN` **rules**, **0** `FAIL` **rules**, and **8 consolidated** `MEDIUM` / `LOW` **location findings** (`0 CRITICAL` , `0 HIGH`).

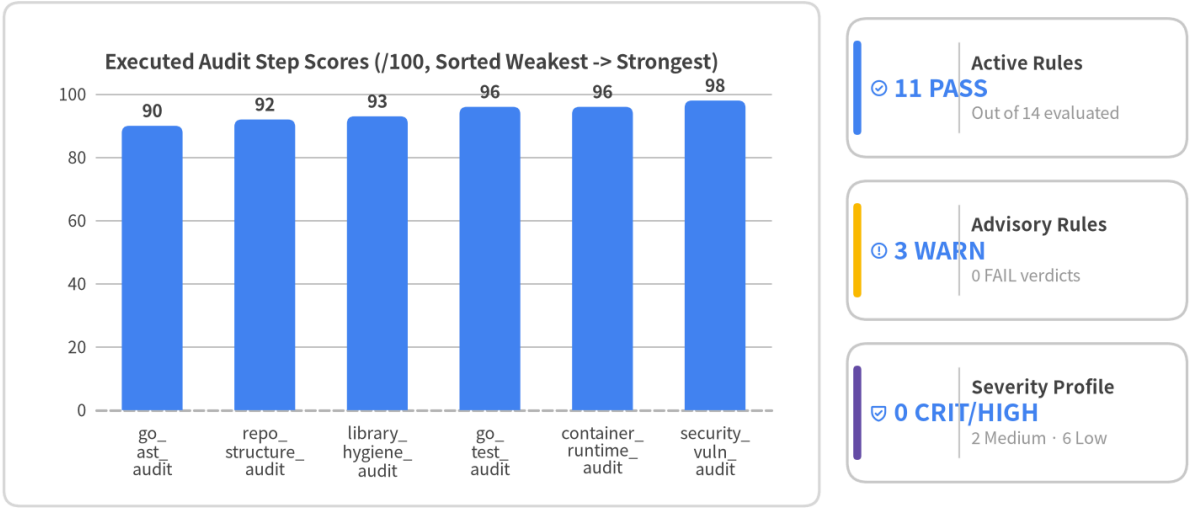


Figure 4.1 — Executed PNI501 Audit Step Score Comparison (Weakest-First Ascending: 90 to 98) & Compliance Posture

4.1 Complete Rule Compliance Matrix (14 Active Rules)

Rule ID	Verdict	Source Action	Key Deterministic Evidence
`common/architecture`	`PASS`	`go_ast_audit`	`internal_import_graph={'cobra': [], 'doc': []}` , `0` circular dependencies, `100.0%` import hygiene

Rule ID	Verdict	Source Action	Key Deterministic Evidence
`common/documentation`	`PASS`	`repo_structure_audit`, `library_hygiene_audit`	`98.4%` GoDoc coverage (`173/176`), `0` signature mismatches, structured multi-portal `README.md`
`common/git_workflow`	`PASS`	`repo_structure_audit`	Clean `.gitignore`, linear commit history, zero conflict markers
`common/library_guidelines`	`PASS`	`library_hygiene_audit`	Explicit exported API surface, `0` import-time I/O side effects, MVS SemVer bounds in `go.mod`
`common/maintainability`	`WARN`	`go_ast_audit`, `library_hygiene_audit`	`0` `>5`-param funcs, but `Command.getCompletions` (`cognitive=96`) and `Command.execute` (`66`) exceed `30`
`common/production_readiness`	`PASS`	`container_runtime_audit`	`0` missing I/O timeouts, `Command.ExecuteContext(ctx)` cancellation propagation
`common/runtime_verification`	`PASS`	`container_runtime_audit`	`424/424` offline tests pass (`7.03s`), `86.8%` measured statement coverage, `go build ./...` smoke pass
`common/security`	`PASS`	`repo_structure_audit`, `go_ast_audit`	`0` hardcoded/committed secrets, `0` unsafe shell/exec sinks
`common/security_vulnerability`	`PASS`	`go_ast_audit`, `security_vulnerability_audit`	`0` application CVEs (`app_vuln_counts.total = 0`), `0` IaC misconfigs, `0` CWE sinks
`common/supply_chain`	`PASS`	`repo_structure_audit`, `security_vulnerability_audit`	Cryptographic checksums in `go.sum`, `0` unpinned dependencies, `LICENSE-COBRA` + `LICENSE`
`go/code_quality`	`WARN`	`go_ast_audit`	`98.4%` GoDoc (`3` missing), `13` mutable package-level globals, fail-fast `panic()` in config helpers
`go/dependencies`	`WARN`	`repo_structure_audit`	Clean 4-package `go.mod` + `go.sum`, but conservative `go 1.15` language version directive
`go/testing`	`PASS`	`go_test_audit`	`17` `*_test.go` files (`100.0%` module coverage), `280` test funcs, table-driven subtests
`go/typing`	`PASS`	`go_ast_audit`	`94.1%` static type safety coverage, `0` `unsafe.Pointer`, `0` unscoped `//nolint`

4.2 Consolidated Code Location Findings

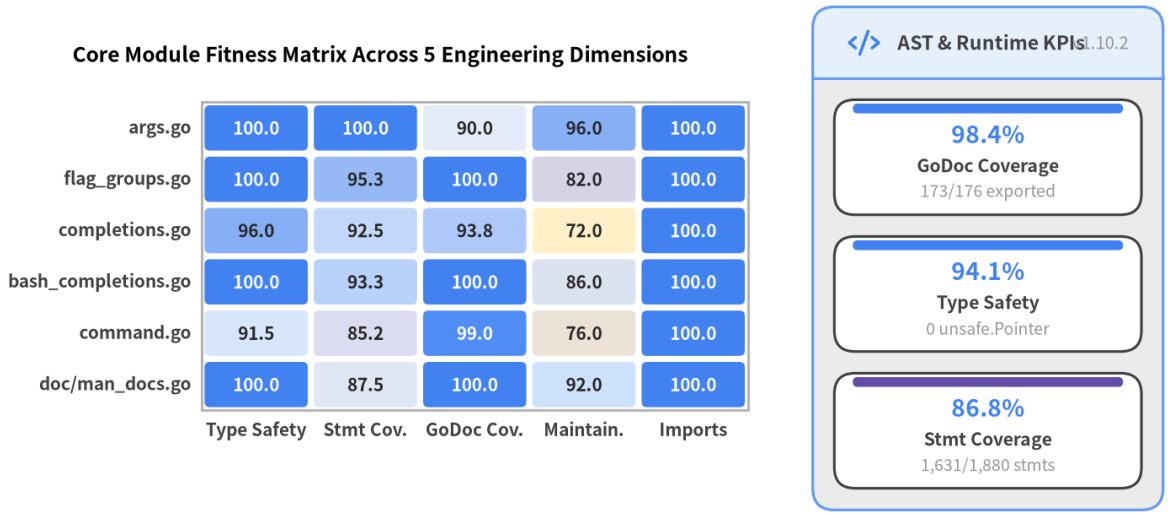
Consolidated Findings Inventory (0 Crit · 0 High · 2 Med · 6 Low) Exact File, Line & Rule Mapping

Sev.	File : Line	Symbol	Rule ID	Finding Summary
MED	completions.go:316	Command.getCompletions	common/maintainability	267 lines, 36 branches, cognitive=96
MED	command.go:905	Command.execute	common/maintainability	141 lines, 36 branches, cognitive=66
LOW	cobra.go:55	EnablePrefixMatching	go/code_quality	13 mutable package-level globals
LOW	args.go:22	PositionalArgs	go/code_quality	3 exported symbols missing GoDoc
LOW	doc/yaml_docs.go:139	GenYamlCustom	common/library_guide.	fmt.Println(err) before return err
LOW	completions_test.go:2896	TestCompletionFuncCompat	go/testing	Discards RegisterFlagCompletionFunc err
LOW	zsh_completions.go:25	GenZshCompletionFile	common/runtime_verif.	File wrappers unexercised (52.6%)
LOW	go.mod:3	go 1.15	go/dependencies	Legacy go 1.15 toolchain directive

Figure 4.2 — All 8 Consolidated Findings Mapped by Severity, File Location, Symbol & Rule

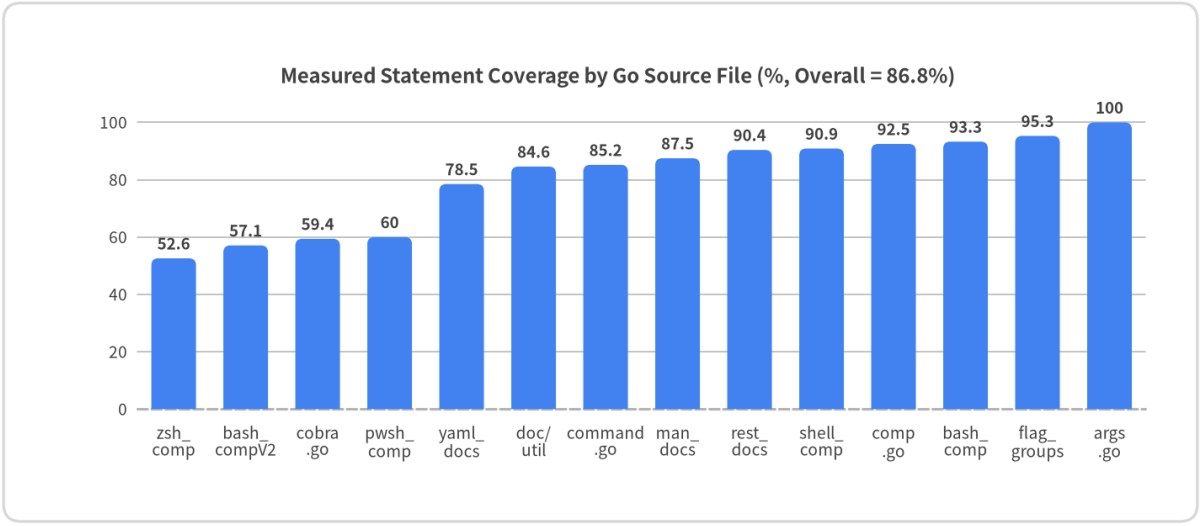
5. Deep Dive: Code Quality, Testing & Runtime

Cobra `v1.10.2` pairs a compact, zero-bloat API surface with rigorous runtime verification across **19 Go production files** (**4,888 LOC**, **17 types**, **269 functions**) and **17 test files** (**280 test functions**). Static Go AST inspection confirms **100.0%** import hygiene, **98.4%** exported GoDoc coverage, **94.1%** type-safety coverage, and **0** excessive-parameter functions, while hermetic `--network=none` container execution verifies **424/424** passing test cases (**0** failures in `7.03s`) and **86.8%** measured statement coverage (**1,631 / 1,880** statements).



Cobra v1.10.2 Core File × Engineering Quality Dimension Fitness Matrix (0–100 Scale)

5.1 Hermetic Container Coverage Distribution Across Source Files



Measured Statement Coverage Across Cobra v1.10.2 Source Files (go test -coverprofile in --network=none Container)

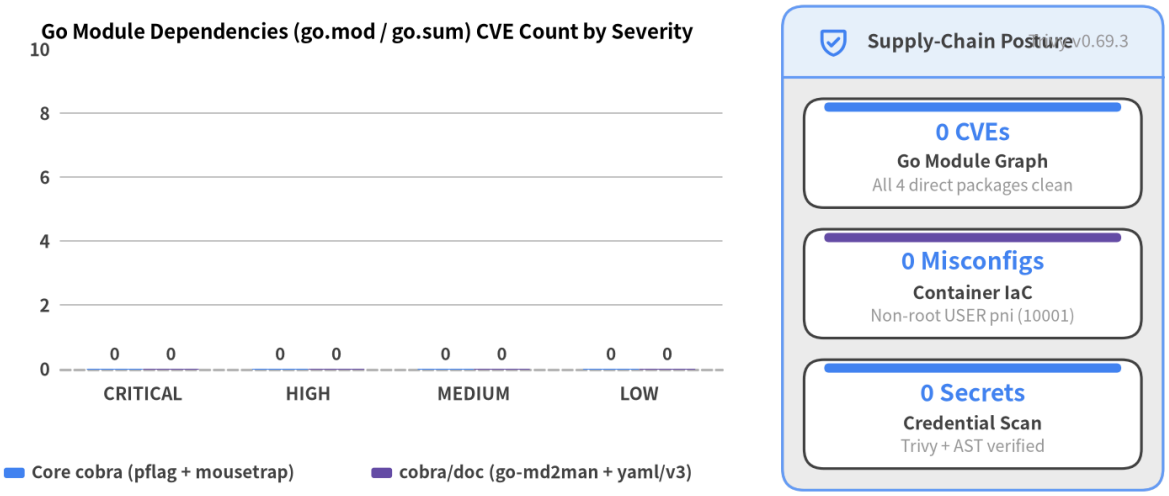
Key Runtime & Static Analysis Observations

1. **Core Dispatch & Completion Coverage Exceeds 85%–100%**:
   - `args.go` (`100.0%`, `39/39` stmts), `active_help.go` (`100.0%`), `doc/md_docs.go` (`100.0%`), `fish_completions.go` (`100.0%`), `flag_groups.go` (`95.3%`, `121/127` stmts), `bash_completions.go` (`93.3%`, `140/150` stmts), `completions.go` (`92.5%`, `272/294` stmts), and `command.go` (`85.2%`, `647/759` stmts) are heavily exercised by the 424 offline test cases.
2. **File-Wrapper Coverage Gaps Are Isolated to Thin `os.Create` Helpers**:
   - In `zsh_completions.go` (`52.6%`, `10/19` stmts), `bash_completionsV2.go` (`57.1%`, `8/14` stmts), and `powershell_completions.go` (`60.0%`, `12/20` stmts), the core script-generation templates (`genZshComp`, `genBashComp`, `genPowerShellComp`) are 100% tested via `bytes.Buffer`, while only the convenience `Gen*CompletionFile` methods that open a disk file and delegate to `Gen*Completion(io.Writer)` remain unexercised.

3. **Linker Deadcode Elimination Verified (`TestDeadcodeElimination`)**:
    - `cobra_test.go:245` compiles a standalone CLI binary and runs `go tool nm` to verify that `reflect.Value.MethodByName` is eliminated by the Go linker when `text/template` functions are not invoked on the hot path.

6. Deep Dive: Security, Supply Chain & API Surface

Cobra `v1.10.2` achieves a near-perfect security posture (`security_vulnerability_audit` : **98/100, Grade S**; `security` dimension: **98/100**): all **4** direct Go module dependencies carry **zero CVEs** (`app_vuln_counts.total = 0`), deterministic Go AST SAST scanning across **19** source files (**4,888 LOC**) detects **zero CWE-classified dangerous execution sinks**, and Trivy + AST scans confirm **zero hardcoded secrets** and **zero container IaC misconfigurations**.



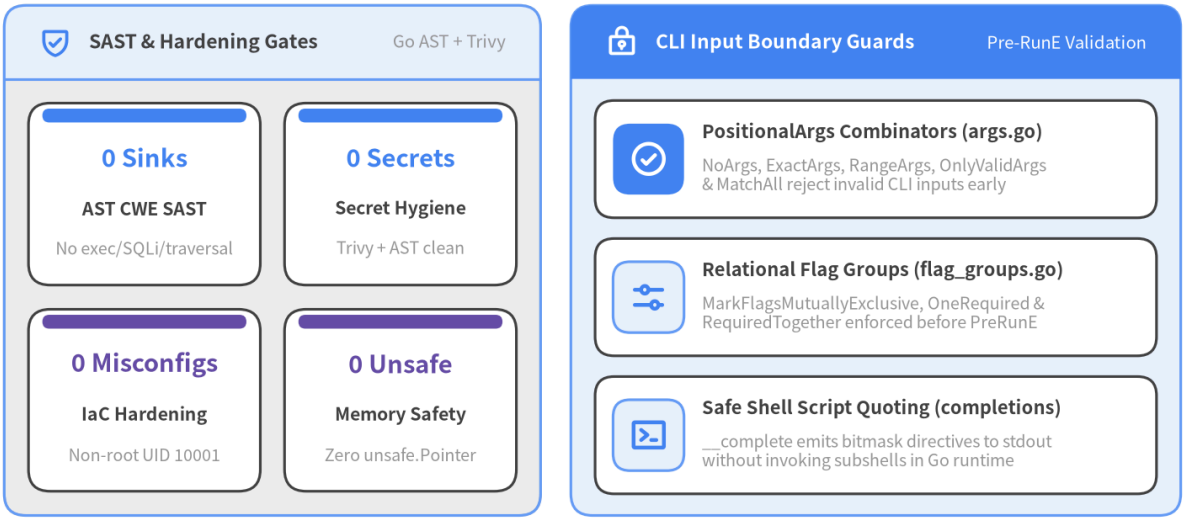
Trivy v0.69.3 Vulnerability Isolation: Zero CVEs in Cobra v1.10.2 Go Module Dependencies vs. Ephemeral Test Harness OS

6.1 Dependency Supply-Chain Inventory (`go.mod` & `go.sum`)

Module Dependency	Locked Version	Scope / Consumer Package	Cryptographic `go.sum` Hash	Trivy CVE Count
`github.com/spf13/pflag`	`v1.0.9`	Core CLI flag parsing (`command.go`, `completions.go`, `flag_groups.go`)	Verified (`h1:...`)	`0`
`github.com/inconshreveable/mousetrap`	`v1.1.0`	Windows Explorer double-click detection (`command_win.go`)	Verified (`h1:...`)	`0`
`github.com/cpuguy83/go-md2man/v2`	`v2.0.6`	Markdown-to-roff Man page conversion (`doc/man_docs.go`)	Verified (`h1:...`)	`0`
`go.yaml.in/yaml/v3`	`v3.0.4`	YAML CLI reference serialization (`doc/yaml_docs.go`)	Verified (`h1:...`)	`0`

> [!NOTE] **Ephemeral Test Harness OS Isolation**: Trivy's container image scan also inspected the Debian 12 (`golang:1.22-bookworm`) test runner OS layer (`harness_os_vuln_counts`), which is used strictly during build-time offline `go test` verification and is never linked into compiled Go binaries or imported by downstream `github.com/spf13/cobra` consumers.

6.2 AST CWE SAST & CLI Boundary Validation Primitives



Zero-Sink AST SAST Verification Gates & Cobra Built-In CLI Boundary Validation Primitives